Faculty of Sciences Departement of Computer Science

Master thesis

Nauty (trouver un meilleur titre)

Auteur: Rodolphe Prévot

Promoteur : Robin Petit

Directeur : Gwenaël Joret

Academic year 2025–2026

Contents

Chapter 1

Introduction

Chapter 2

Preliminaries

This chapter introduces all the definitions necessary to understand this document. For further details on the following concepts, refer to Reinhard Diestel's book on graph theory [1].

2.1 Algebra Definitions

Definition 1 (informal)**.** An *isomorphism* is a one-to-one correspondence (mapping) between two sets that preserves binary relationships between elements of the sets.

An *automorphism* is an isomorphism from a set to itself.

Definition 2. A *group* is a set with a binary operation that satisfies the following constraints: the operation is associative, it has an identity element, and every element of the set has an inverse. For example, $(\mathbb{R}, +)$ is a group: it consists of the set of real numbers with addition as the binary operation. It has as identify element: 0, every element has an inverse element (each number with its opposite) and it's assoc iative: for all numbers a, b, c in $\mathbb{R}$: $a + (b + c) = (a + b) + c$.

A *homomorphism* is a transformation of one set into another that preserves in the second set the relations between elements of the first. For example, given two groups, $(G, \cdot)$ and $(H, \circ)$, a *group homomorphism* from $(G, \cdot)$ to $(H, \circ)$ is a function $\phi : G \to H$ such that $\phi(g_1 \cdot g_2) = \phi(g_1) \circ \phi(g_2)$ for every $g_1, g_2 \in G_1$.

Let $(G, *)$ be a group with identity element e, and let X be a non-empty set. A *group action* of G on X exists if there is a map

$$\cdot : G \times X \to X, \quad (g, x) \mapsto g \cdot x$$

such that the following two properties hold:

1. Identity: $\forall x \in X, \quad e \cdot x = x$,
2. Compatibility: $\forall g, h \in G,\ \forall x \in X, \quad (g * h) \cdot x = g \cdot (h \cdot x)$.

As an example, let S_n denote the *symmetric group* acting on a set X. We indicate the action of elements of S_n by exponentiation: for $x \in X$ and $g \in S_n$, x^g denotes the image of x under g. The same notation is used to indicate the induced action on complex structures derived from X. As additional notation, for $W \subseteq V$, we write $W^g = \{w^g : w \in W\}$.

Definition 3. Let a group G act on a set X, and let $S \subseteq X$. The *pointwise stabilizer* of S in G is the subgroup defined by:

$$G_{(S)} = \{g \in G \mid \forall x \in S,\ g \cdot x = x\}.$$

Definition 4. An *equivalence relation*, denoted by $\sim$, on a set X is a binary relation that is reflexive, symmetric, and transitive.

For any element $x \in X$, the *equivalence class* of x under $\sim$ is the subset

$$[x] = \{y \in X \mid y \sim x\}.$$

The *quotient set* of X by $\sim$, denoted by

$$X/\sim = \{[x] \mid x \in X\},$$

is the set of all equivalence classes of X under $\sim$.

2.2 Graph Definitions

Definition 5. A *graph* is a pair $G = (V, E)$ of sets satisfying $E \subseteq \binom{V}{2}$; thus, the elements of E are 2-element subsets of V. The elements of V are the *vertices* of the graph G, the elements of E are its *edges*.

Let V^* denote the set of finite sequences of vertices. For $\sigma \in V^*$, $|\sigma|$ denotes the number of components of σ.If $\sigma = (v_1, v_2, \ldots, v_k) \in V^*$ and $w \in V$ then $\sigma \mid\mid w$ denotes $(v_1, v_2, \ldots, v_k, w)$. Furthermore, for $0 \leq s \leq k$,
$[\sigma]_s \coloneqq (v_1, v_2, \ldots, v_s)$.

An acyclic graph (one not containing any cycles) is called a *forest*. A connected forest is called a *tree*. Each vertex of a tree is called a *node*, and a node of degree 1 (only connected to one node) is called a *leaf*. When a particular node is designated as the starting point, that node is called the *root*.

Let $G = (V, E)$ be a graph. Let $\sim$ be an equivalence relation on V. The *quotient graph* of G with respect to $\sim$, denoted $G/\sim$, is a graph (V', E') such that:

1. $V' = V/\sim = \{[v] \mid v \in V\}$
2. $E' = \{[u][v] \mid uv \in E\}$

Definition 6. We let $\mathcal{G}_n$ be the set of all graphs on n vertices (without loss of generality, say $V = \{1, 2, \ldots, n\}$).$\mathcal{G}$ represents the set of all graphs.

2.2.1 Definitions specifics to isomorphisms

Definition 7. A *colouring* of V (or of $G \in \mathcal{G}$) is a surjective function π from V onto $\{1, 2, \ldots, k\}$ for some k. The number of colours, i.e. k, is denoted by $|\pi|$. A cell of π is the set of vertices with some given colour; that is, $\pi^{-1}(j)$ for some j with $1 \leq j \leq |\pi|$. We let Π be the set of all colourings.

If π, π' are colourings, then π' is *finer than or equal* to π, written $\pi' \preccurlyeq \pi$, if $\pi(v) < \pi(w) \Rightarrow \pi'(v) < \pi'(w)$ for all $v, w \in V$ (This implies that each cell of π' is a subset of a cell of π).

Here, the colouring is not related to a *proper colouring*, π is related to a *partitioning*.

A *discrete colouring* is a colouring in which each cell is a singleton, in which case $|\pi| = n$.

A *coloured graph* is a pair (G, π) where π is a colouring of G.

A colouring of a graph is called *equitable* if any two vertices of the same colour are adjacent to the same number of vertices of each colour. This is also called an *equitable/stable partition*.

Definition 8. Let S_n denote the *symmetric group* acting on V.

There are several notations to consider:

1. if π is a colouring of V, then π^g is the colouring with $\pi^g(v^g) = \pi(v)$ for each $v \in V$.
2. if (G, π) is a coloured graph, then $(G, \pi)^g = (G^g, \pi^g)$.

These properties show that two coloured graphs (G, π) and (G', π') are isomorphic iff there exists $g \in S_n$ such that $(G', \pi') = (G, \pi)^g$. In this case, we write $(G, \pi) \cong (G', \pi')$.

$\mathrm{Aut}(G, \pi) = \{g \in S_n : (G, \pi)^g = (G, \pi)\}$; $\mathrm{Aut}(G, \pi)$ is the *Automorphism group* of coloured graph (G, π).

Definition 9. The *canonical form* is a function

$$C : \mathcal{G} \times \Pi \to \mathcal{G} \times \Pi$$

such that for all $G \in \mathcal{G}, \pi \in \Pi$ and $g \in S_n$:

1. $C(G, \pi) \cong (G, \pi)$.
2. $C(G^g, \pi^g) = C(G, \pi)$.

Chapter 3

Algorithme Nauty

In this section, we will focus extensively on McKay's general algorithm: Nauty.

3.1 Introduction de l'algorithme

L'algorithme de Nauty va se baser sur la forme canonique pour arriver à son but final: trouver une labellisation canonique d'un graphe, et ainsi trouver son groupe d'automorphisme. Mais concrètement, pourquoi est-ce que la forme canonique est si importante? En quoi est-ce que c'est un outil puissant pour trouver une labellisation canonique d'un graphe?

La forme canonique va permettre de coutourner la difficulté du problème d'isomorphisme: plutôt que de comparer deux graphes entre eux, on associe à chaque graphe un représentant unique et standard, appelé graphe canonique (ou canonical form). Deux graphes sont alors isomorphes si et seulement si leurs formes canoniques sont identiques. Cela transforme donc un problème d'équivalence en un problème d'égalité, ce qui est, beaucoup plus facile à résoudre. Cependant, Comment Nauty arrive à la construire? Comment est-ce que l'algorithme arrive à trouver cette forme canonique? C'est ce que nous allons expliquer dans les sections suivantes.

Nous allons donc commencer par introduire l'algorithme même et expliquer les différentes fonctions qui le composent, ainsi que la manière de trouver la forme canonique du graphe. Ensuite, nous parlerons de la construction du groupe d'automorphisme à partir de la forme canonique.

3.2 Search Tree construction

First of all, the algorithm itself is built upon a very specific type of tree: a search tree.

De manière informel, un search tree est un arbre de recherche dans lequel, chaque noeud est un état, et chaque branche va correspondre à une action faite qui sera donc différente pour chaque branche. Ça va nous permettre d'explorer tous les états possibles à partir d'un état initial, et de trouver l'état final que nous cherchons. Nous faisons donc bien de la recherche sur un arbre, d'où le nom de search tree.

However, its particularity is that the leaves of this tree correspond to sequences of vertices of a coloured graph. As we move down the tree, these sequences grow longer. These sequences define a colouring of the graph obtained by first assigning unique colours to the vertices in the sequence and then deducing, in a "controlled" manner, the colouring of the remaining vertices. The leaves of this tree correspond to sequences whose derived colouring is discrete.The root of this tree is therefore the "empty" sequence.

To correctly construct this tree and find our canonical form, we will need several functions:

- *A refinement function*: Starting from a partition, this function allows us to construct a new partition that is finer than the previous one.
- *A target cell selection function*: Starting from a partition, this function allows us to choose a cell from this partition in order to create the "child" of the previous partition.
- *A node invariant function*: A function that allows us to efficiently compare the different nodes/leaves of our search tree in order to determine the canonical form of the graph.

These three functions are the fundamental components required to construct our search tree and find our canonical form. Before defining them formally, we will informally explain how they work and how they interact with each other to build our search tree.

Starting from a graph, we first apply the refinement function to obtain a so-called equitable partition (we will come back to this later when discussing the implementation of this function). This partition constitutes the root of our search tree.

Then, as long as we do not have a discrete partition, we apply the target cell selector to choose a cell from the current partition. Each branch represents a choice of element from the parent node's selected cell. This cell is then split into two cells using an individualisation method (which will also be detailed

later). For each branch, we apply the refinement function again to obtain a new partition, thereby creating a new node. Finally, on each node, we apply the node invariant function, which will be useful for determining the canonical form of our graph. We repeat this process until we obtain discrete partitions, which correspond to the leaves of our search tree. In summary, each node represents a partition, and each branch represents a cell chosen from the parent node to be individualised.

However, there exist additional functions that are more specific to the implementation of the algorithm and that will be explained in the implementation section of this document.

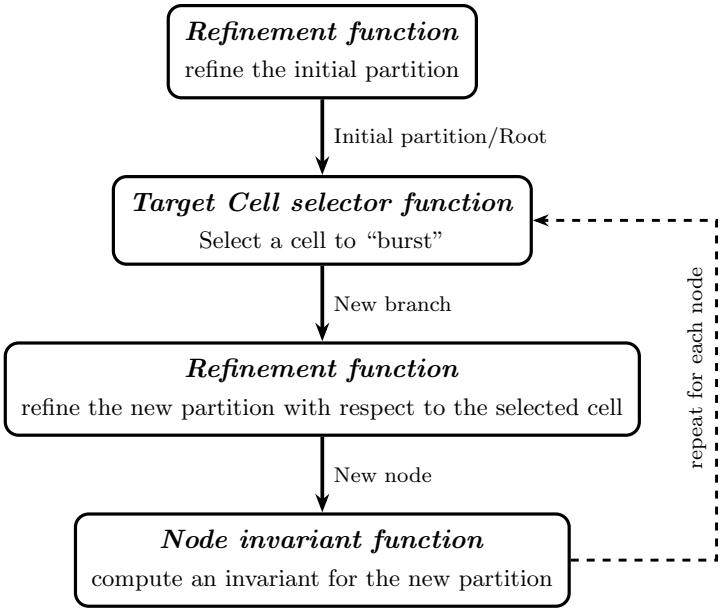

Figure 3.1: Construction du *search tree* dans *Nauty*.

Maintenant, que nous avons bien mis en place les différentes fonctions qui composent notre algorithme, nous allons pouvoir les définir formellement et établir les propriétés de notre search tree.

3.2.1 Formal definitions of the search tree functions

Definition 10. The *Refinement function* is a function

$$R : \mathcal{G} \times \Pi \times V^* \to \Pi$$

such that, for all $G \in \mathcal{G}, \pi \in \Pi$ and $\sigma \in V^*$,

R1. $R(G, \pi, \sigma) \preccurlyeq \pi$.

R2. if $\sigma_i \in \sigma$, then $\{\sigma_i\}$ is a cell of $R(G, \pi, \sigma)$.

R3. for any $g \in S_n$, we have $R(G^g, \pi^g, \sigma^g) = R(G, \pi, \sigma)^g$.

This means that:

1. the refinement function returns a colouring that is finer than π.
2. if a vertex (v) belongs to the vertex sequence (σ), then it must belong to a cell of size 1 in the refined colouring.
3. if we take a permutation of our graph, our colouring, and our vertex sequence, and then apply the refinement function to these elements, we obtain the same result as if we first applied the refinement function to our graph, our colouring, and our vertex sequence, and then applied the same permutation to the resulting colouring.

Definition 11. The *Target cell selector function* is a function

$$T : \mathcal{G} \times \Pi \times V^* \to 2^V$$

such that, for all $G \in \mathcal{G}, \pi \in \Pi$ and $\sigma \in V^*$

T1. if $R(G, \pi_0, \sigma)$ is discrete, then $T(G, \pi_0, \sigma) = \emptyset$.

T2. if $R(G, \pi_0, \sigma)$ is not discrete, then $T(G, \pi_0, \sigma)$ is a non-singleton cell of $R(G, \pi_0, \sigma)$.

T3. for any g $\in S_n$, we have $T(G^g, \pi^g, \sigma^g) = T(G, \pi, \sigma)^g$.

This means that:

1. if the refinement function returns a discrete partition (a leaf of our search tree), then the *target cell selection function* must not return any cell, i.e., it returns $\emptyset$.
2. if the refinement function returns a non-discrete partition (a node of our search tree), then the *target cell selection function* returns a cell of this partition that contains more than one element.
3. if we take a permutation of our graph, our colouring, and our vertex sequence, and then apply the *target cell selection function* to these elements, we obtain the same result as if we first applied the *target cell selection function* to our graph, our colouring, and our vertex sequence, and then applied the same permutation to the resulting cell.

Now that all the formal definitions of the tree have been established, we can define the search tree $\mathcal{T}(G, \pi_0)$, which depends on an initial coloured graph (G, π_0). All the nodes of the tree are elements of V^*. This search tree has some characteristics to keep in mind:

- The root of $\mathcal{T}(G, \pi_0)$ is the empty sequence ().

- if σ is a node of $\mathcal{T}(G, \pi_0)$, let W $= T(G, \pi_0, \sigma)$. Then the children of π are $\{\sigma \mid\mid w : w \in W\}$.
- a node σ of $\mathcal{T}(G, \pi_0)$ is a leaf iff $R(G, \pi_0, \sigma)$ is discrete.
- for any node σ of $\mathcal{T}(G, \pi_0)$, define $\mathcal{T}(G, \pi_0, \sigma)$ to be the subtree of $\mathcal{T}(G, \pi_0)$ consisting of σ and all its descendants.

[**Rodolphe:** Rédiger une explication non-formelle sur les propriétés]

To demonstrate the correctness of the algorithm, the pruning procedure that we will introduce, and to prove that we indeed obtain the canonical form of the graph, we need to establish several properties of this search tree.

Lemma 1. *For any $G \in \mathcal{G}, \pi_0 \in \Pi, g \in S_n$ we have $\mathcal{T}(G^g, \pi_0^g) = \mathcal{T}(G, \pi_0)^g$.*

Proof. Let $\sigma = (\sigma_1, \ldots, \sigma_k)$ be a node of $\mathcal{T}(G, \pi_0)$. We first show that $\mathcal{T}(G, \pi_0)^g \subseteq \mathcal{T}(G^g, \pi_0^g)$. Such that $\sigma^g = (\sigma_1^g, \ldots, \sigma_k^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

We proceed by induction on s, for $0 \leq s \leq k$, showing that $[\sigma^g]_s = (\sigma_1^g, \ldots, \sigma_s^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

Base case ($s = 0$): The empty sequence () is the root of any search tree. So $[\sigma^g]_0$ is trivially a node of $\mathcal{T}(G^g, \pi_0^g)$.

Inductive step: Assume $[\sigma^g]_s = (\sigma_1^g, \ldots, \sigma_s^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$. Since $(\sigma_1, \ldots, \sigma_s, \sigma_{s+1})$ is a node of $\mathcal{T}(G, \pi_0)$, the element σ_{s+1} was individualised to go from level s to level $s+1$. Since we apply g to the entire structure, σ_{s+1} is also mapped to σ_{s+1}^g, which is equivalent to directly individualising σ_{s+1}^g in G^g with π_0^g. Hence $[\sigma^g]_{s+1} = (\sigma_1^g, \ldots, \sigma_s^g, \sigma_{s+1}^g)$ is a node of $\mathcal{T}(G^g, \pi_0^g)$.

By induction, σ^g is a node of $\mathcal{T}(G^g, \pi_0^g)$, hence $\mathcal{T}(G, \pi_0)^g \subseteq \mathcal{T}(G^g, \pi_0^g)$.

The reverse inclusion follows by applying the same argument with g^{-1} in place of g, which gives $\mathcal{T}(G^g, \pi_0^g)^{g^{-1}} \subseteq \mathcal{T}(G, \pi_0)$, and therefore $\mathcal{T}(G^g, \pi_0^g) \subseteq \mathcal{T}(G, \pi_0)^g$. □

Corollary 2. *Let σ be a node of $\mathcal{T}(G, \pi_0)$ and let $g \in Aut(G, \pi_0)$. Then σ^g, is a node of $\mathcal{T}(G, \pi_0)$ and $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$.*

Proof.
Part 1: σ^g is a node of $\mathcal{T}(G, \pi_0)$.
By Lemma 1, we have:

$$\mathcal{T}(G^g, \pi_0^g) = \mathcal{T}(G, \pi_0)^g$$

Since $g \in \mathrm{Aut}(G, \pi_0)$, we have $G^g = G$ and $\pi_0^g = \pi_0$, and therefore:

$$\mathcal{T}(G, \pi_0) = \mathcal{T}(G, \pi_0)^g$$

Since σ is a node of $\mathcal{T}(G, \pi_0)$, σ^g is a node of $\mathcal{T}(G, \pi_0)^g = \mathcal{T}(G, \pi_0)$.

Part 2: $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$
We apply Lemma 1 to the subtree rooted at σ. Since g maps each node of the subtree rooted at σ to a node of the subtree rooted at σ^g, and since $g \in \mathrm{Aut}(G, \pi_0)$ implies $G^g = G$ and $\pi_0^g = \pi_0$, the same argument as in Lemma 1 gives both inclusions:

$$\mathcal{T}(G, \pi_0, \sigma)^g \subseteq \mathcal{T}(G, \pi_0, \sigma^g) \quad \text{and} \quad \mathcal{T}(G, \pi_0, \sigma^g) \subseteq \mathcal{T}(G, \pi_0, \sigma)^g$$

and therefore $\mathcal{T}(G, \pi_0, \sigma^g) = \mathcal{T}(G, \pi_0, \sigma)^g$. □

Lemma 3. *Let σ be a node of (G, π_0) and let $\pi = R(G, \pi_0, \sigma)$. Then $\mathrm{Aut}(G, \pi)$ is the point-wise stabilizer of σ in $\mathrm{Aut}(G, \pi_0)$.*

Proof. We show both inclusions to establish the equality.

Part 1: $\mathrm{Aut}(G, \pi) \subseteq$ point-wise stabilizer of σ
Let $g \in \mathrm{Aut}(G, \pi)$. By R2, for each $\sigma_i \in \sigma$, $\{\sigma_i\}$ is a singleton cell of $\pi = R(G, \pi_0, \sigma)$. Since g is an automorphism of (G, π), it must map each cell of π to a cell of π. A singleton $\{\sigma_i\}$ can only be mapped to itself, hence $\sigma_i^g = \sigma_i$ for every $\sigma_i \in \sigma$, so g stabilizes σ point-wise.

Part 2: point-wise stabilizer of $\sigma \subseteq \mathrm{Aut}(G, \pi)$
Let $g \in \mathrm{Aut}(G, \pi_0)$ be such that g stabilizes σ point-wise, that is $\sigma^g = \sigma$. We want to show that $g \in \mathrm{Aut}(G, \pi)$. So, $\pi^g = \pi$. By R3 applied to G, π_0 and σ:

$$R(G^g, \pi_0^g, \sigma^g) = R(G, \pi_0, \sigma)^g$$

Since $g \in \mathrm{Aut}(G, \pi_0)$, we have $G^g = G$ and $\pi_0^g = \pi_0$, and since g stabilizes σ point-wise, $\sigma^g = \sigma$. Therefore:

$$R(G, \pi_0, \sigma) = R(G, \pi_0, \sigma)^g$$

which gives exactly $\pi = \pi^g$, and hence $g \in \mathrm{Aut}(G, \pi)$. □

Now that we have properly defined our search tree, we can discuss the node invariant function and how it allows us to compute the canonical form of the graph.

Definition 12. Let Ω be some totally ordered set. A *Node invariant* is a function

$$\phi : \mathcal{G} \times \Pi \times V^* \to \Omega$$

Such that, for any $\pi_0 \in \Pi$, $G \in \mathcal{G}$ and distinct $\sigma, \sigma' \in \mathcal{T}(G, \pi_0)$.

The function ϕ has a few properties to take into account:

1. if $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) < \phi(G, \pi_0, \sigma')$,
   then for every leaf $\sigma_1 \in \mathcal{T}(G, \pi_0, \sigma)$ and leaf $\sigma_1' \in \mathcal{T}(G, \pi_0, \sigma')$ we have $\phi(G, \pi_0, \sigma_1) < \phi(G, \pi_0, \sigma_2)$.

2. if $\pi = R(G, \pi_0, \sigma)$ and $\pi' = R(G, \pi_0, \sigma')$ are discrete,
   then $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma') \Leftrightarrow G^{\pi} = G^{\pi'}$ (equality relation).
3. for any $g \in S_n$, we have $\phi(G^g, \pi_0^g, \sigma^g) = \phi(G, \pi_0, \sigma)$.
4. Leaves σ, σ' are *equivalent* if $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, \sigma')$. This is the case, if there is a unique $g \in Aut(G, \pi_0)$ such that $\sigma^g = \sigma'$, with $g = R(G, \pi_0, \sigma')R(G, \pi_0, \sigma)^{-1}$.

This means that:

1. if two vertex sequences are equal and ϕ is strictly smaller with σ than σ', then this inequality propagates to all descendant leaves: no leaf descending from σ can exceed any leaf descending from σ'.
2. if we have two equal colourings, then their invariants are equal if and only if they produce exactly the same canonical labelling; applying the permutation associated with their colouring yields exactly the same graph.
3. if we take a permutation of our graph, our colouring, and our vertex sequence, and then apply the *node invariant* to these elements, it is equivalent to applying the *node invariant* directly to our original graph, colouring, and vertex sequence.
4. [**Rodolphe:** Faire explication 4]

We can define the canonical form:

$$\phi^*(G, \pi_0) = \max \{\phi(G, \pi_0, \sigma) : \sigma \text{ is a leaf of } \mathcal{T}(G, \pi_0)\},$$

3.2.2 Implementation strategies

Now that we have all the theoretical tools in hand, we can begin discussing the implementation of the algorithm. Our implementation is based on the definitions of the algorithm, but it is not a direct or simplistic reproduction of its source code.

Implementation Refinement function

In order to ensure that the properties of the refinement function defined in the search tree are respected, we must implement an algorithm that outputs a colouring that is equal to (or finer) than the original colouring. To this end, we rely on a theorem: for every colouring π, there exists a unique (up to cell ordering) coarsest equitable colouring π' such that $\pi' \preccurlyeq \pi$.

To construct our refinement function, we need two auxiliary functions: a function that refines a colouring so that it becomes an equitable colouring,

and an individualisation function.

Definition 13. The function $F(G, \pi, \alpha)$ is our refinement algorithm (implemented in a label-invariant manner).

pseudo-code of $F(G, \pi, \alpha)$ (from [4])

```
1: Data: π is the input colouring and α is a sequence of some cells of π
2: Result: the final value of π is the output colouring (equitable)
3: while α is not empty and π is not discrete do
4:     Remove some element W from α
5:     for each cell X of π do
6:         Let X_1, ..., X_k be the fragments of X distinguished according
           to the number of edges from each vertex to W
7:         Replace X by X_1, ..., X_k in π
8:         if X ∈ α then
9:             Replace X by X_1, ..., X_k in α
10:        else
11:            Add all but one of the largest of X_1, ..., X_k to α
12:        end if
13:    end for
14: end while
```

[**Rodolphe:** Faire une explication plus détaillée de l'algorithme?]

Definition 14. The *individualized function* $I : \Pi \times V \to \Pi$

such that, if v is a vertex in a non-singleton cell of π and $\pi' = I(\pi, v)$, then for $w \in V$,

$$\pi'(w) = \begin{cases} \pi(w), & \text{if } \pi(w) < \pi(v) \text{ or } w = v; \\ \pi(w) + 1, & \text{otherwise.} \end{cases}$$

We say that the vertex v is individualized in π. We see that $I(\pi, v)$ differs from π in that a unique colour has been given to vertex v.

[**Rodolphe:** Même chose ici? Faire une explication plus détaillée de la fonction?]

Now we can define a refinement function. For a sequence of vertices v_1,v_2,..., define

- $R(G, \pi_0, ()) = F(G, \pi_0,$ a list of all the cells of $\pi_0)$
- $R(G, \pi_0, (v_1)) = F(G, I((R(G, \pi_0, ()), v_1), \{v_1\}))$
- $R(G, \pi_0, (v_1, v_2)) = F(G, I((R(G, \pi_0, (v_1)), v_2), \{v_2\}))$
- $R(G, \pi_0, (v_1, v_2, v_3)) = F(G, I((R(G, \pi_0, (v_1, v_2)), v_3), \{v_3\}))$

- and so on.

The algorithm satisfies the required properties of a refinement function and performs well in practice. However, the refinement step is the most computationally expensive part of the algorithm, which is why special care must be taken in its implementation. C'est pourquoi, pour réduire les temps de calculs, nous allons utiliser des méthodes de prunning dont nous parlerons un peu plus tard.

Implementation Target Cell Selector

[**Rodolphe:** Même chose ici, c'est un peu du copier coller du papier de mckay, je laisse ça comme ça, ou alors je rajoute un peu de sel avec?]

[**Rodolphe:** Mettre pseudo-code]

For the implementation of the target cell selection function in the construction of the search tree, several strategies can be adopted. Choosing the right target cell at each step is important because it influences the shape and depth of the tree, and therefore the overall performance of the search.

In the original version of *Nauty*, McKay recommended selecting the first smallest non-singleton cell, as it increases the chance that the selected cell is part of an orbit of the group that stabilizes the current sequence [3]. However, later studies showed that in certain cases, it is better to simply select the first non-singleton cell, regardless of its size [2].

As a result, *Nauty* now supports two strategies for target cell selection that we can choose:

1. Always select the first smallest non-singleton cell.
2. Select the first cell that is joined in a non-trivial way to the largest number of other cells, where a non-trivial join means the number of edges between two cells is strictly between 0 and the maximum possible.

In our implementation we will only use the first strategy.

[**Rodolphe:** S'il reste du temps, intéressant de faire une comparaison entre les 2 stratégies?]

Implementation Node Invariant

Now that the search tree is implemented, we need to define the node invariant function, which will allow us to determine the canonical form. To compute this function, we can rely on two related sources:

1. For each node σ, there is a colouring $R(G, \pi_0, \sigma)$. We can use its properties to extract the number and size of the cells, as well as combinatorial properties of the coloured graph.

2. We can use the intermediate states of the colouring computation from the parent node (in the refinement algorithm), such as the position and size of the cells produced during the refinement procedure, and various edge counts determined during the computation. If $f(\sigma)$ is a function of this information—computed during the calculation of $R(G, \pi_0, \sigma)$ and from the resulting coloured graph—then the vector $(f([\sigma]_0), f([\sigma]_1), \ldots, f(\sigma))$, with lexicographic ordering, satisfies the characteristics required for a node invariant. In *Nauty*, the second source is used: the value of $f(\sigma)$ is an integer, and the pruning rules are applied as each node is computed.

Another potential method is based on the concept of the *quotient graph* $Q(G, \pi)$ (see Definition 5), assuming that the colouring π of graph G is equitable. All vertices of $Q(G, \pi)$ are cells of π, and they include information about the size and number of the cell. For any two cells $C_1, C_2 \in \pi$, possibly equal, the corresponding vertices in $Q(G, \pi)$ are connected by an edge labelled with the number of edges in G between C_1 and C_2. The node invariant already computed by *Nauty* is a deterministic function of the sequence of quotient graphs $Q(G, R(G, \pi_0, [v]_i))$ for $i = 0, \ldots, |v|$. Therefore, it would be possible to use the sequence of quotient graphs to obtain information about the cells, but this would be far too costly in both time and memory.

In our implementation, we will use the first source described above. We rely on the number and size of the cells, and we apply combinatorial properties of the coloured graph to compute our node invariant. Several invariant methods have been implemented in order to test their efficiency and computational speed. We will present the different methods we implemented, as well as the results obtained for each of them, in the results section.

[**Rodolphe:** Bon je savais pas trop quoi dire sur la fin, dans le sens où je sais pas si je présente déjà ici mes méthodes d'invariants, (j'ai ici que le nombre de triangle dans les cellules mais je rajouterai les prochaines que vais implémenter.), ou alors comme j'ai dit, je mets dans plus dans les résultats mais en faisant ce commentaire j'ai l'impression que c'est quand même mieux ici]

We will also briefly discuss the computational complexity of these different invariant methods, and how they compare to one another. Since this function is called at each node creation, it is important that its computation remains fast in order to avoid slowing down the algorithm.

Here is the exhaustive list of invariant methods that we have implemented so far:

1. *Number of triangles per cell*: for each cell of the partition, we compute the number of triangles contained in that cell. We then obtain a vector where each entry corresponds to the number of triangles in the corresponding cell. Finally, we apply a lexicographic ordering to this vector to compute our node invariant.

   For counting the number of triangles in a cell, several methods are possible; the most classical one runs in $O(n^3)$, but there also exist faster approaches in [**Rodolphe:** Pas encore codé mais ça arrive pour avoir ma complexité].

   pseudo-code of

   1: **Data:**
   2: **Result:**

Implementation prunning

To avoid generating unnecessary information in our search tree,which would result in an overly large tree, we aim to construct the most optimized tree possible. To achieve this, we can apply *pruning* techniques to the tree: eliminating redundant branches so that only a fraction of the original tree needs to be generated. We can define two types of pruning strategies to optimize our construction:

$P_A(\sigma, \sigma')$: Suppose σ, σ' are distinct nodes of $T(G, \pi_0)$ with $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) > \phi(G, \pi_0, \sigma')$. Then, we will remove $T(G, \pi_0, \sigma')$.

$P_B(\sigma, \sigma')$: Suppose σ, σ' are distinct nodes of $T(G, \pi_0)$ with $|\sigma| = |\sigma'|$ and $\phi(G, \pi_0, \sigma) \neq \phi(G, \pi_0, \sigma')$. Then, we will remove $T(G, \pi_0, \sigma')$.

[**Rodolphe:** J'ai retiré PC vu que ce n'est pas encore implémenté, mais je le rajouterais quand on parlera de aut (2ème partie donc)]

Despite these pruning strategies, we must ensure that our search tree, along with the node invariant function, remains functionally correct. To this end, we require a theorem that guarantees the validity of the overall construction.

Theorem 4. *Consider any $G \in \mathcal{G}$ and $\pi_0 \in \Pi$*

- *Suppose any sequence of operations of the form $P_A(\sigma, \sigma')$ or $P_C(\sigma, g)$ are performed.*
  Then there remains at least one leaf σ_1 with $\phi(G, \pi_0, \sigma_1) = \phi^*(G, \pi_0)$.
- *Let σ_0 be some fixed leaf of $T(G, \pi_0)$. Suppose any sequence of operations of the form $P_B(\sigma, \sigma')$ $P_C(\sigma'', g)$ are performed,*
  where $\phi(G, \pi_0, \sigma'') \neq \phi(G, \pi_0, [\sigma_0]_{|\sigma''|})$. Let $g_1, \ldots, g_k$ be the automorphisms used in the operations P_C that were performed, and let

$A = \{g \in \mathrm{Aut}(G, \pi_0)$*:* v_0^g *is a remaining leaf*$\}$*. Then* $\mathrm{Aut}(G, \pi_0)$ *is generated by* $\{g_1, \ldots, g_k\} \cup A$*.*

[**Rodolphe:** Faudrait que je refasse la preuve ici par moi-même mais ça parle de Pc, est-ce que je retirais la preuve, je ferai une mini preuve pour expliquer que même avec Pa et PB c'est good, et puis je ferai une mini preuve pour Pc quand on parlera du groupe d'automorphisme?]

Implementation of the search tree

We can put in practice ours different pruning strategies on our search tree. *Nauty* will generate its tree in depth-first order. The lexicographically least leaf σ_1 is kept and if the canonical labelling is sought (rather than just the automorphism group), the leaf σ^* with the greatest invariant discovered so far is also kept. A non-discrete node σ is pruned if neither $\phi(G, \pi_0, \sigma) = \phi(G, \pi_0, [\sigma_1]_{|\sigma|})$ or $\phi(G, \pi_0, \sigma) \geq \phi(G, \pi_0, [\sigma^*]_{|\sigma|})$. Such operations have both type $P_A(\sigma^*, \sigma)$ and $P_V(\sigma_1 *, \sigma)$.[**Rodolphe:** Ça parle aussi de P_C ici, est-ce que j'enlève?]

Bibliography

[1] R. Diestel. *Graph Theory*, volume 173 of *Graduate Texts in Mathematics*. Springer, Heidelberg, 6th edition, 2025. ISBN: 978-3-662-53621-6.

[2] A. Kirk. Efficiency considerations in the canonical labelling of graphs. Technical report, Technical report TR-CS-85-05, Computer Science Department, Australian, 1985.

[3] B. D. McKay et al. Practical graph isomorphism, 1981.

[4] B. D. McKay and A. Piperno. Practical graph isomorphism, II. *Journal of symbolic computation*, 60:94–112, 2014.